

Masked Language Models

Meng Jiang¹

¹Department of Computer Science and Engineering (CSE)
University of Notre Dame

CSE 60556 LLM

Table of Contents

1 Neural LMs

2 BERT

3 Extensions

Table of Contents

1 Neural LMs

2 BERT

3 Extensions

Autoregressive Language Modeling

The probability of observing a sequence X is:

$$P(X) = \prod_{i=1}^{|X|} P(x_i | x_1, \dots, x_{i-1}). \quad (1)$$

Next token prediction is to predict

$$P(x_i | x_1, \dots, x_{i-1}). \quad (2)$$

Count-based Unigram Models

Independence assumption:

$$P(x_i | x_1, \dots, x_{i-1}) \approx P(x_i). \quad (3)$$

Parameterizing in log space: Multiplication of probabilities can be re-expressed as addition of log probabilities.

$$P(X) = \prod_{i=1}^{|X|} P(x_i) \Rightarrow \log P(X) = \sum_{i=1}^{|X|} \log P(x_i). \quad (4)$$

We define these "parameters":

$$P_{opt}(x \in V) = \frac{c(x)}{\sum_{x' \in V} c(x')}, \quad (5)$$

for each word x in the vocabulary V ; $c(x)$ is the word count.

Higher-order n -gram Models

Limit context length to n , count, and divide:

$$P(x_i | x_{i-n+1}, \dots, x_{i-1}) = \frac{c(x_{i-n+1}, \dots, x_{i-1}, x_i)}{c(x_{i-n+1}, \dots, x_{i-1})}. \quad (6)$$

Strengths:

- Extremely fast to estimate/apply.

Weaknesses:

- Cannot share strength among similar words, e.g., "buy a car" vs "purchase a car";
- Cannot handle long-distance dependencies.

Feed-forward neural LMs

Learning $P(x_i | x_{i-n+1}, \dots, x_{i-1})$:

- Lookup for token embeddings: $x_{i-n+1} \Rightarrow \mathbf{x}_{i-n+1}, \dots, x_{i-1} \Rightarrow \mathbf{x}_{i-1}$;
- Concatenate: $\mathbf{x}_{i-n+1:i-1}$;
- Non-linear transformation: $\mathbf{h}_i := f(\mathbf{W} \cdot \mathbf{x} + \mathbf{b})$;
- Lookup for tokens: $\mathbf{h}_i \Rightarrow x_i$.

Think:

- How is strength among similar words shared?
- How many parameters are there in $\mathbf{W}$? Is it bigger or smaller than "word count parameters" $\{c(\cdot)\}$?

Modeling Long-distance Dependencies is Challenging

- Agreement in number, gender, etc.

***He** does not have very much confidence in **himself**.*

***She** does not have very much confidence in **herself**.*

- Selectional preference:

*The **reign** has lasted as long as the life of the **queen**.*

*The **rain** has lasted as long as the life of the **clouds**.*

- Complicated coreference:

*The **trophy** would not fit in the suitcase because **it** was too **big**.*

*The trophy would not fit in the **suitcase** because **it** was too **small**.*

Sequence Modeling to Learn Dependencies

Three types to pass token signals through layers:

- Recurrence: Representations conditioned on an encoding of history;
- Convolution: Representations conditioned on tokens in local context;
- Attention: Representations conditioned on a weighted average of all tokens in the context.

Think:

- Suppose the former layer gives $\mathbf{x}_1, \dots, \mathbf{x}_4$. Can you make diagrams to differentiate how they are processed to the signals on the current layer $\mathbf{h}_1, \dots, \mathbf{h}_4$ using the three different designs?

Recurrent Neural Networks

Advantage compared against feed-forward: $\mathbf{h}_i = f(\mathbf{W}_h \mathbf{h}_{i-1} + \mathbf{W}_x \mathbf{x}_i + \mathbf{b})$.

Weakness: Vanishing gradients caused by non-linear function f and small weights in $\mathbf{W}$. In the worst case, this may completely stop the network from further training.

Solution: Long Short-Term Memory (LSTM) Structure.

Readings:

- Mikolov 2010: Recurrent neural network based language model.
- Karpathy 2015: The Unreasonable Effectiveness of RNNs.

Attention

Suppose we have a token-embedding matrix $\mathbf{X} \in \mathbb{R}^{n \times k}$ from the former layer, where n is the sequence length and k is the embedding size. We use matrices $\mathbf{W}_Q \in \mathbb{R}^{k \times d_k}$, $\mathbf{W}_K \in \mathbb{R}^{k \times d_k}$, and $\mathbf{W}_V \in \mathbb{R}^{k \times k}$ to have the "query vector", "key vector", and "value vector" of each token:

$$\mathbf{Q} = \mathbf{XW}_Q \in \mathbb{R}^{n \times d_k}, \mathbf{K} = \mathbf{XW}_K \in \mathbb{R}^{n \times d_k}, \mathbf{V} = \mathbf{XW}_V \in \mathbb{R}^{n \times k}. \quad (7)$$

Then we calculate attention scores – weights between tokens:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}, \quad (8)$$

and the attention output on the current layer is:

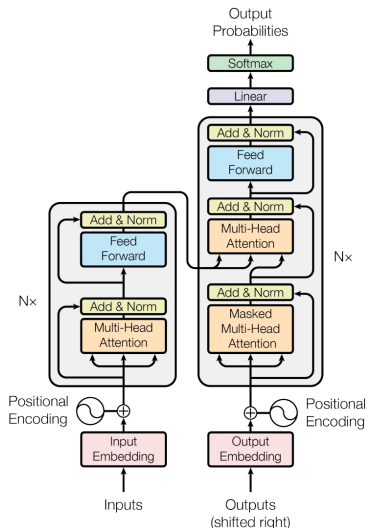
$$\mathbf{H} = \mathbf{AV} \in \mathbb{R}^{n \times k}. \quad (9)$$

Training attention layers is to learn the three weight matrices.

Attention and Transformer

Readings:

- **Cross attention:** Bahdanau et al. 2014: Neural Machine Translation by Jointly Learning to Align and Translate.
- **Self attention:** Cheng et al. 2016: Long Short-Term Memory-Networks for Machine Reading.
- **Transformer:** Vaswani et al. 2017: Attention is All You Need.



Novel Design in Transformer

- **Multi-head attention:** e.g., one head for **syntax patterns** (nearby context) and the other head for **semantics** (farther context):

*I **run** a small **business**.*

*I **run** a **mile** in 10 minutes.*

*The **robber** made a **run** for it.*

*The tech **stock** had a **run**.*

- **Positional encodings:** distinguishing identical words? add an embedding based on word positions!
*a **big** dog and a **big** cat*
- **Layer normalization:** Vanishing gradients in RNNs also occur in multi-layer transformers. Make an additive connection between the input and output: "**Add & Norm**".
 - Ba et al. 2016: Layer Normalization.

Transformer: Main Results

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

from Vaswani et al. 2017: Attention is All You Need.

Table of Contents

1 Neural LMs

2 BERT

3 Extensions

What is BERT?

BERT: Bidirectional Encoder Representations from Transformers.
(GPT: Generative Pre-trained Transformer.)

BERT models are **Pre-trained** models.

They both use a **transfer learning** approach: pre-training + fine-tuning.

- Training a model from scratch is expensive because the parameters are randomly initialized.
- Pre-training is to gain base knowledge. The parameters are getting close to what target tasks need.
- Fine-tuning is to further optimize the parameters for the target tasks.

Pre-training Tasks

Goal: tune the parameters to predict accurate contextual (sequence) representations for natural language **understanding**.

Task 1: Masked Language Modeling (MLM): understand context to predict words.

- Solution: Mask out $k\%$ (usually $k=15$) of the input words, and then predict the masked words.

Task 2: Next Sentence Prediction (NSP): understand the relationship between sentences.

- Solution: Introduce [SEP], a special token used to separate two segments. Sample two contiguous segments for 50% of the time and random segments for another 50%, and then do binary prediction.

Training BERT

Input embeddings = Token embeddings + Segment embeddings + Position embeddings

Pre-trained jointly on MLM and NSP.

Fine-tuned on token-level and sentence-level tasks.
(what are MNLI, NER, SQuAD?)

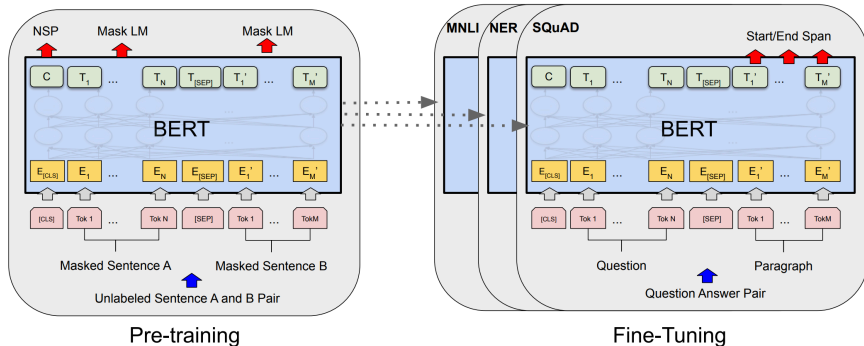


Table of Contents

1 Neural LMs

2 BERT

3 Extensions

SciBERT: Developed by AllenAI

Trained BERT-base on a random sample of **1.14M papers** from Semantic Scholar: 18% papers from the computer science domain (CS) and 82% from the broad biomedical domain (Bio).

The average paper length is 154 sentences resulting in a corpus size of **3.17B tokens**, similar to the 3.3B tokens on which BERT-base was trained.

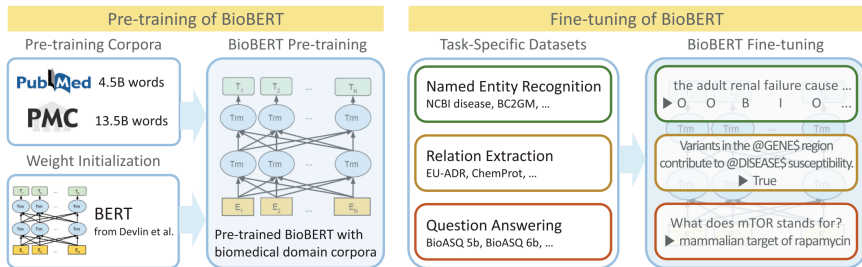
from Beltagy et al. 2019: SciBERT: A Pre-trained Language Model for Scientific Text

SciBERT: Results

Field	Task	Dataset	SOTA	BERT-Base		SciBERT	
				Frozen	Finetune	Frozen	Finetune
Bio	NER	BC5CDR (Li et al., 2016)	88.85 ⁷	85.08	86.72	88.73	90.01
		JNLPBA (Collier and Kim, 2004)	78.58	74.05	76.09	75.77	77.28
		NCBI-disease (Dogan et al., 2014)	89.36	84.06	86.88	86.39	88.57
	PICO	EBM-NLP (Nye et al., 2018)	66.30	61.44	71.53	68.30	72.28
	DEP	GENIA (Kim et al., 2003) - LAS	91.92	90.22	90.33	90.36	90.43
		GENIA (Kim et al., 2003) - UAS	92.84	91.84	91.89	92.00	91.99
	REL	ChemProt (Kringelum et al., 2016)	76.68	68.21	79.14	75.03	83.64
CS	NER	SciERC (Luan et al., 2018)	64.20	63.58	65.24	65.77	67.57
	REL	SciERC (Luan et al., 2018)	n/a	72.74	78.71	75.25	79.97
	CLS	ACL-ARC (Jurgens et al., 2018)	67.9	62.04	63.91	60.74	70.98
Multi	CLS	Paper Field	n/a	63.64	65.37	64.38	65.71
		SciCite (Cohan et al., 2019)	84.0	84.31	84.85	85.42	85.49
Average				73.58	77.16	76.01	79.27

Named Entity Recognition (NER), PICO Extraction (PICO),
Text Classification (CLS), Relation Classification (REL),
Dependency Parsing (DEP)

BioBERT



- BioBERT v1.0: 1M steps on Wiki/Books + 200K steps on PubMed + 270K steps on PMC;
- BioBERT v1.1: + **1M steps** on PubMed.

from Lee et al. 2019: BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining

BioBERT: NER (and many other) Data for Fine-tuning

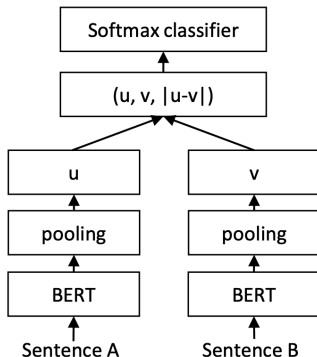
Dataset	Entity type	Number of annotations
NCBI Disease (Doğan <i>et al.</i> , 2014)	Disease	6881
2010 i2b2/VA (Uzuner <i>et al.</i> , 2011)	Disease	19 665
BC5CDR (Li <i>et al.</i> , 2016)	Disease	12 694
BC5CDR (Li <i>et al.</i> , 2016)	Drug/Chem.	15 411
BC4CHEMD (Krallinger <i>et al.</i> , 2015)	Drug/Chem.	79 842
BC2GM (Smith <i>et al.</i> , 2008)	Gene/Protein	20 703
JNLPBA (Kim <i>et al.</i> , 2004)	Gene/Protein	35 460
LINNAEUS (Gerner <i>et al.</i> , 2010)	Species	4077
Species-800 (Pafilis <i>et al.</i> , 2013)	Species	3708

BioBERT: Results

Type	Datasets	Metrics	BERT		BioBERT v1.0			BioBERT v1.1
			SOTA	(Wiki + Books)	(+ PubMed)	(+ PMC)	(+ PubMed + PMC)	(+ PubMed)
Disease	NCBI disease	P	<u>88.30</u>	84.12	86.76	86.16	89.04	88.22
		R	<u>89.00</u>	87.19	88.02	89.48	<u>89.69</u>	91.25
		F	<u>88.60</u>	85.63	87.38	87.79	<u>89.36</u>	89.71
	2010 i2b2/VA	P	<u>87.44</u>	84.04	85.37	85.55	87.50	86.93
		R	<u>86.25</u>	84.08	85.64	85.72	85.44	86.53
		F	86.84	84.06	85.51	85.64	86.46	<u>86.73</u>
	BC5CDR	P	89.61	81.97	85.80	84.67	85.86	<u>86.47</u>
		R	83.09	82.48	86.60	85.87	<u>87.27</u>	87.84
		F	<u>86.23</u>	82.41	86.20	85.27	86.56	87.15
Drug/chem.	BC5CDR	P	94.26	90.94	92.52	92.46	93.27	<u>93.68</u>
		R	92.38	91.38	92.76	92.63	93.61	<u>93.26</u>
		F	93.31	91.16	92.64	92.54	<u>93.44</u>	93.47
	BC4CHEMD	P	<u>92.29</u>	91.19	91.77	91.65	<u>92.23</u>	92.80
		R	90.01	88.92	<u>90.77</u>	90.30	90.61	91.92
		F	91.14	90.04	91.26	90.97	<u>91.41</u>	92.36
Gene/protein	BC2GM	P	81.81	81.17	81.72	82.86	85.16	<u>84.32</u>
		R	81.57	82.42	83.38	84.21	83.65	<u>85.12</u>
		F	81.69	81.79	82.54	83.53	<u>84.40</u>	84.72
	JNLPBA	P	74.43	69.57	71.11	71.17	<u>72.68</u>	72.24
		R	<u>83.22</u>	81.20	83.11	82.76	83.21	83.56
		F	78.58	74.94	76.65	76.53	<u>77.59</u>	77.49
Species	LINNAEUS	P	<u>92.80</u>	91.17	91.83	91.62	93.84	90.77
		R	94.29	84.30	84.72	85.48	<u>86.11</u>	85.83
		F	93.54	87.60	88.13	88.45	<u>89.81</u>	88.24
	Species-800	P	74.34	69.35	70.60	71.54	<u>72.84</u>	72.80
		R	<u>75.96</u>	74.05	75.75	74.71	<u>77.97</u>	75.36
		F	<u>74.98</u>	71.63	73.08	73.09	75.31	74.06

Sentence-BERT

How to fine-tune a BERT model to learn strong sentence embeddings on almost 1 million NLI sentence pairs?



from Reimers et al. 2019: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

Takeaways

- Challenges in language modeling
- Neural LMs: Recurrence, Convolution, Attention
- Transformer's novel design
- BERT pre-training and fine-tuning
- SciBERT, BioBERT, and Sentence-BERT